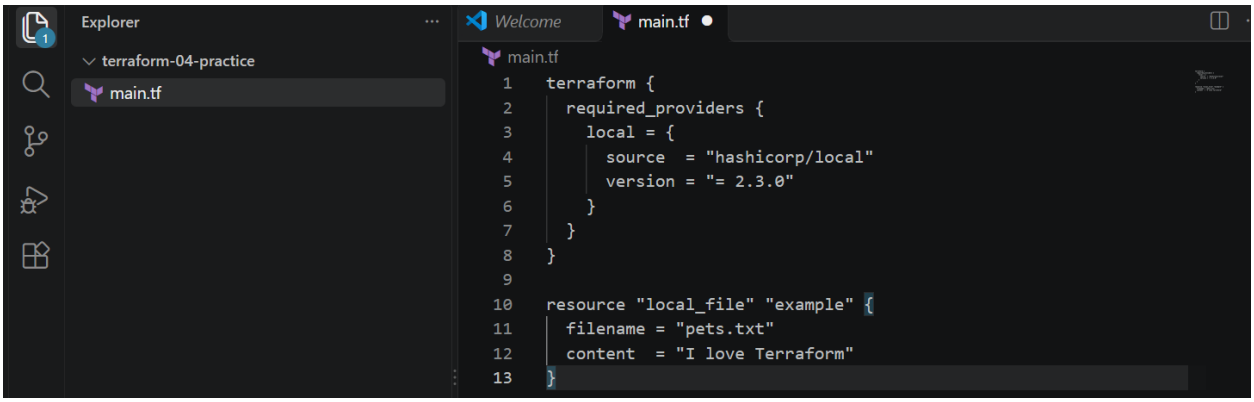
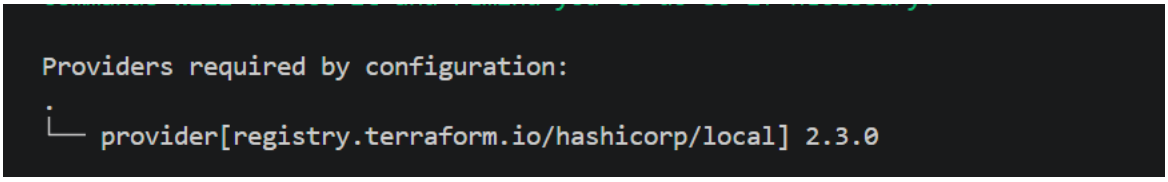
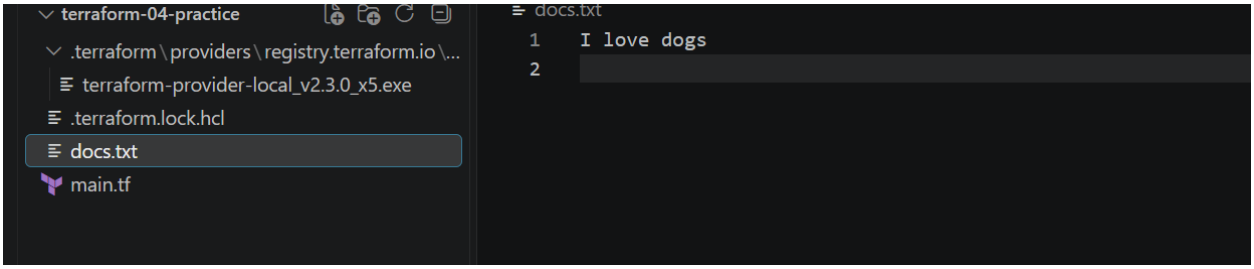
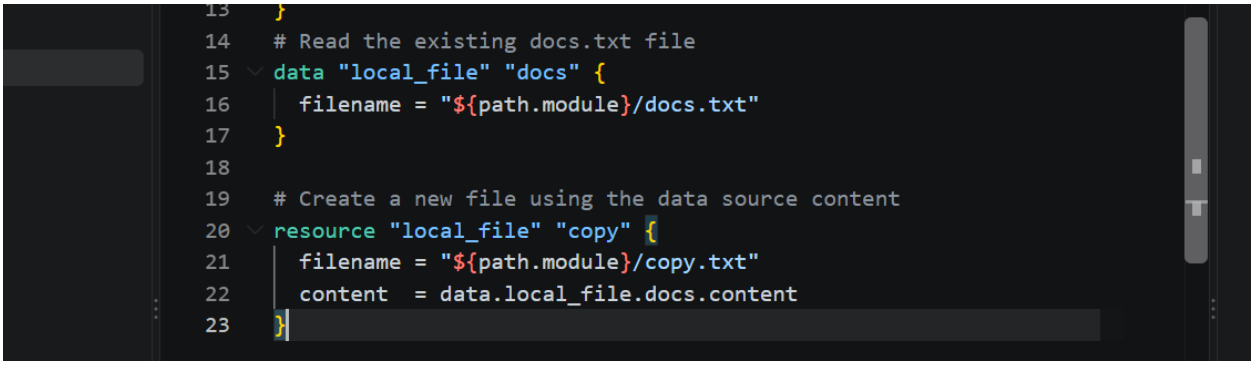
Terraform – 4









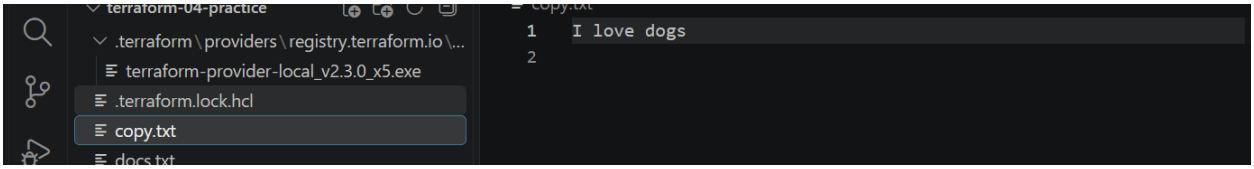
Keep your existing code. Add the following blocks below it:

- **`data "local_file"` reads the existing `docs.txt`.**
- **`resource "local_file"` creates a new file called `copy.txt`.**
- **`data.local_file.docs.content` reads the content from `docs.txt` and passes it into `copy.txt`.**

```
local_file.example: Creating...
local_file.copy: Creating...
local_file.example: Creation complete after 0s [id=f0702396afaf73d3f0a3d14376a3196589381e01]
local_file.copy: Creation complete after 0s [id=34af7eaf089b3b2aba04a8969538892c20bf948d]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
PS C:\Users\india\OneDrive\Desktop\terraform-04-practice>
```

After successful execution, check whether `copy.txt` was created and contains `I love dogs`.



Terraform `count` meta-argument:

```hcl
variable "file_names" {
  type    = list(string)
  default = ["apple.txt", "banana.txt", "orange.txt"]
}

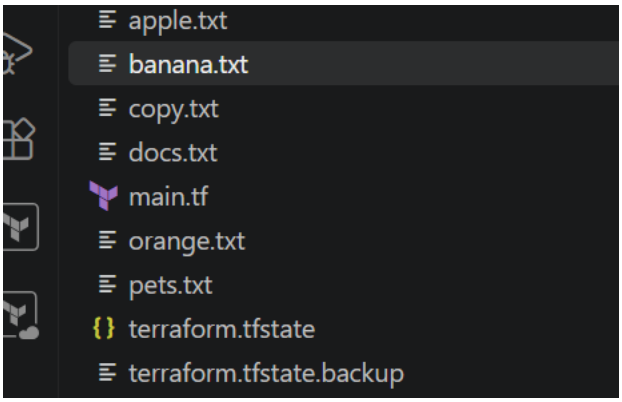
resource "local_file" "fruits" {
  count    = length(var.file_names)
  filename = "${path.module}/${var.file_names[count.index]}"
  content  = "Created using Terraform count"
}
```

Add this at the bottom of the main file

- **`count` creates multiple resources.**
- **`length(var.file_names)` calculates the number of files (3).**
- **`count.index` gives each file an index starting from `0`.**

```
local_file.fruits[2]: Creating...
local_file.fruits[1]: Creating...
local_file.fruits[0]: Creating...
local_file.fruits[0]: Creation complete after 0s [id=7405b241e29f5e8603c57e953e88
a145118d4e9e]
local_file.fruits[1]: Creation complete after 0s [id=7405b241e29f5e8603c57e953e88
a145118d4e9e]
local_file.fruits[2]: Creation complete after 0s [id=7405b241e29f5e8603c57e953e88
a145118d4e9e]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
PS C:\Users\india\OneDrive\Desktop\terraform-04-practice>
```



apple, banana, orange created

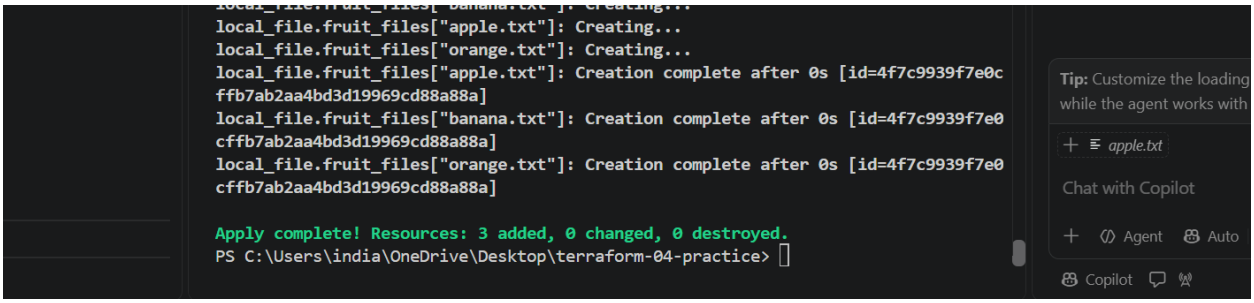
for_each:

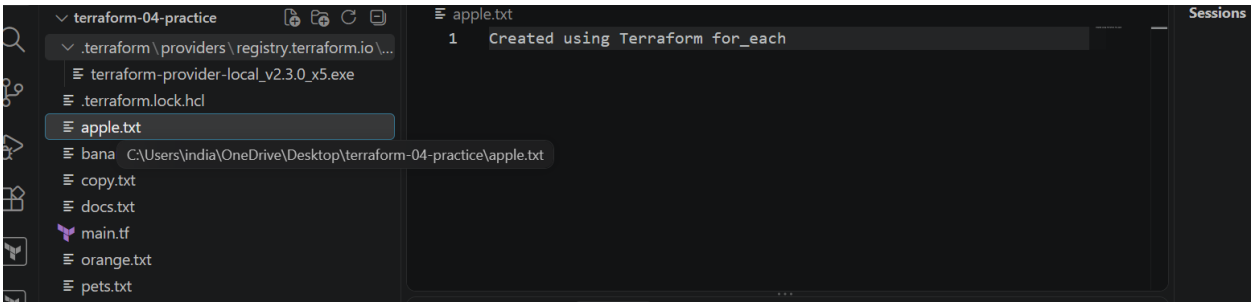
Add this code at the bottom of `main.tf`

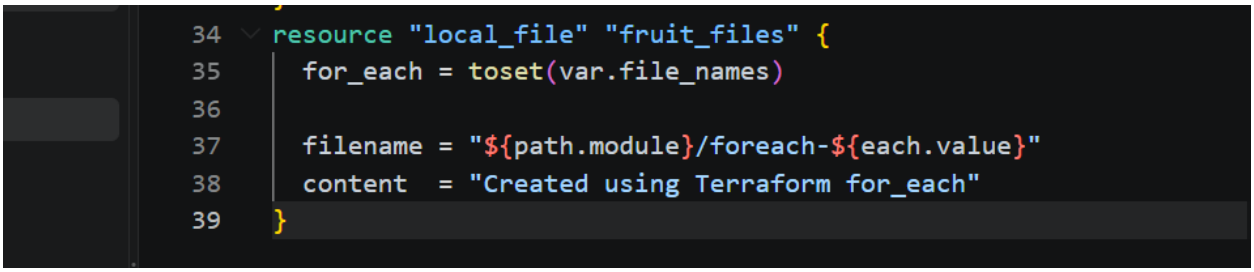
```
resource "local_file" "fruit_files" {
  for_each = toset(var.file_names)

  filename = "${path.module}/${each.value}"
  content  = "Created using Terraform for_each"
}
```

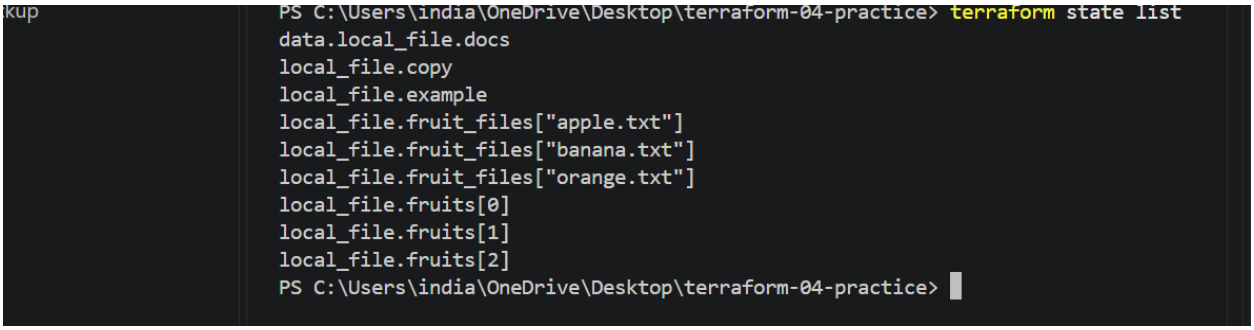
- **`count` uses numerical indexes: `0, 1, 2`.**
- **`for_each` uses each item as its key/value.**
- **`toset()` converts the list into a set.**
- **`each.value` refers to the current filename.**







replace for each





:

Configure AWS provider:



- `required_providers`: Downloads the AWS provider.
- `source`: Official AWS provider from HashiCorp.
- `version`: Specifies the provider version range.
- `provider "aws"`: Configures AWS.
- `us-east-1`: North Virginia region.

```
PS C:\Users\india\OneDrive\Desktop\terraform-04-practice> aws ec2 describe-vpcs --region us-east-1 --query "Vpcs[].VpcId" --output text
vpc-019405835908e8978

PS C:\Users\india\OneDrive\Desktop\terraform-04-practice> aws ec2 describe-subnets --filters "Name=vpc-id,Values=vpc-019405835908e8978" --region us-east-1 --query
```

```
PS C:\Users\india\OneDrive\Desktop\terraform-04-practice> aws ec2 describe-subnets --filters "Name=vpc-id,Values=vpc-019405835908e8978" --region us-east-1 --query "Subnets[].{SubnetId:SubnetId,AZ:AvailabilityZone,CIDR:CidrBlock}" --output table
-------------------------------------------------------------
|                      DescribeSubnets                      |
+------------+---------------+---------------------------+
|     AZ     |     CIDR      |         SubnetId          |
+------------+---------------+---------------------------+
|  us-east-1b|  10.0.2.0/24  |  subnet-04a0839a233b0012c  |
|  us-east-1a|  10.0.1.0/24  |  subnet-0bdb51aaa475e2d85  |
|  us-east-1b|  10.0.12.0/24 |  subnet-0f5bf4cbe60a4a9a8  |
|  us-east-1a|  10.0.11.0/24 |  subnet-030e36f43142f5598  |
-- More  --
```

Create an IAM user using Terraform

```
resource "aws_iam_user" "terraform_user" {

  name = "terraform04-demo-user"

}
```

```
# aws_iam_user.terraform_user will be created
+ resource "aws_iam_user" "terraform_user" {
    + arn           = (known after apply)
    + force_destroy = false
    + id            = (known after apply)
    + name          = "terraform04-demo-user"
    + path          = "/"
    + tags_all      = (known after apply)
    + unique_id     = (known after apply)
  }
```

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "iam:ListUsers",
                "iam:CreateUser",
                "iam:GetUser",
                "iam:DeleteUser",
                "iam:ListGroupsForUser",
                "iam:PutUserPolicy",
                "iam:AttachUserPolicy"
            ],
            "Resource": "*"
        }
    ]
}
```

Edit statement

Add actions

Choose a service

Search services

Included

IAM

Available

```
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_iam_user.terraform_user: Destroying... [id=terraform04-demo-user]
aws_iam_user.terraform_user: Destruction complete after 1s
aws_iam_user.terraform_user: Creating...
aws_iam_user.terraform_user: Creation complete after 1s [id=terraform04-demo-user
]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\Users\india\OneDrive\Desktop\terraform-04-practice>
```

terraform04-demo-user	/	0	-	-	-	

user created

resource "aws_iam_user_policy_attachment" "attach_policy" {

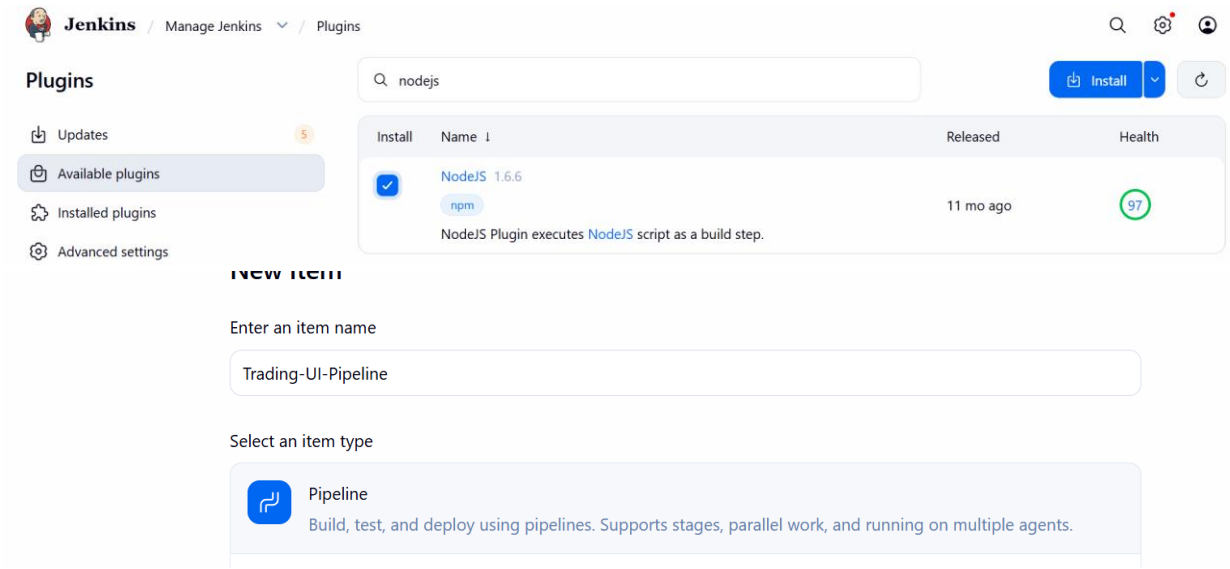
user = aws_iam_user.terraform_user.name

policy_arn = "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"

}

Add this under main file to attach policy

Install NodeJS plugin in Jenkins,




```
pipeline {
  agent any

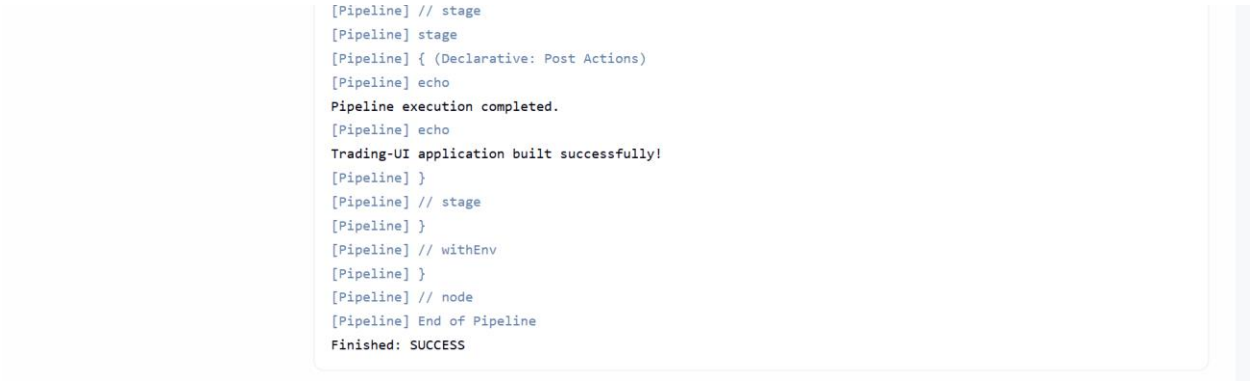
  tools {
    nodejs 'NodeJS-20'
  }

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main',
          url: 'https://github.com/betawins/Trading-UI.git'
      }
    }

    stage('Install Dependencies') {
      steps {
        sh 'npm install'
      }
    }
```

```
    stage('Build') {
      steps {
        sh 'npm run build'
      }
    }
  }
}
```

add this script



```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline execution completed.
[Pipeline] echo
Trading-UI application built successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Important Maven Commands

Maven is a Java build automation and project management tool. It helps compile code, run tests, package applications, and deploy artifacts.

1. `mvn clean`

Deletes the previously generated build files from the `target` directory.

2. `mvn validate`

Checks whether the project structure and configuration are correct.

3. `mvn compile`

Compiles the Java source code.

4. `mvn test`

Runs unit tests to check whether the application works correctly.

5. `mvn package`

Packages the application into a JAR or WAR file.

6. `mvn verify`

Checks whether the package meets the project's verification requirements.

7. `mvn install`

Installs the generated JAR/WAR into the local Maven repository.

8. `mvn deploy`

Deploys the generated artifact to a remote Maven repository, such as Nexus.

9. `mvn dependency:tree`

Displays all project dependencies in a tree structure.

10. `mvn help:effective-pom`

Displays the final POM after combining the project's POM with inherited configurations.